



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Projeto e Análise de
Algoritmos
Professor: Daniel Mendes Barbosa

Trabalho Prático 1 - Labirinto

5084 - Ana Lúcia de Souza
5062 - Gustavo Eufrausino de Medeiros
4705 - Gabriel Santos Ferreira de Pádua
5093 - Matheus Kauan Passos de Souza

SUMÁRIO

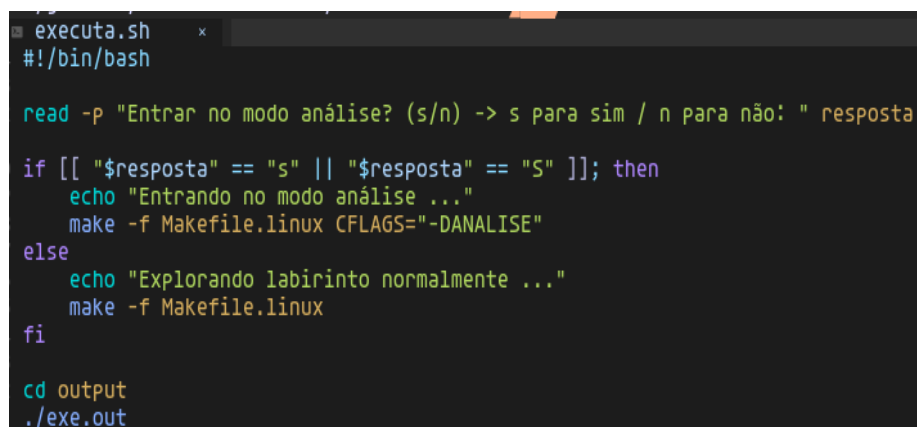
Introdução.....	2
Arquivos e compilação.....	2
Organização:.....	3
/arquivos.....	4
Desenvolvimento:.....	4
Estruturas e Tipos de Dados Abstratos.....	4
Mapa e Leitura.....	6
Menu.....	11
Backtracking.....	14
Modo análise.....	17
Extras.....	19
Ponto extra 3:.....	19
Ponto extra 2:.....	21
Ponto extra 1:.....	22
Ponto Extra 5:.....	22
Ponto Extra 4:.....	24
Resultados.....	25
Conclusão.....	28
Observações:.....	28
Referência:.....	28

Introdução

Este trabalho aborda um desafio específico que combina técnicas de algoritmos de backtracking com escape de labirintos. O problema é inspirado em um cenário fictício em que estudantes de Ciência da Computação precisam escapar de labirintos. Os labirintos são representados por arquivos de texto, contendo informações sobre a posição inicial, obstáculos e condições especiais de movimentação, como o uso de chaves para liberar passagens, em formato de uma matriz. O objetivo desse trabalho é projetar e implementar um algoritmo de backtracking que explore todas as possibilidades de movimentos para encontrar uma solução viável, se houver.

Arquivos e compilação

Para a compilação e execução do código-fonte deste trabalho prático foi implementado um arquivo de script em bash chamado “executa.sh”. Esse arquivo consiste em: receber uma entrada do usuário e, a depender da resposta, entrar ou não no modo análise; compilar o programa com o arquivo “Makefile.linux”; mudar para o diretório onde o executável “exe.out” foi gerado e, por fim, executar esse novo arquivo. A figura 1 ilustra o corpo de arquivo de script.



```
executa.sh
#!/bin/bash

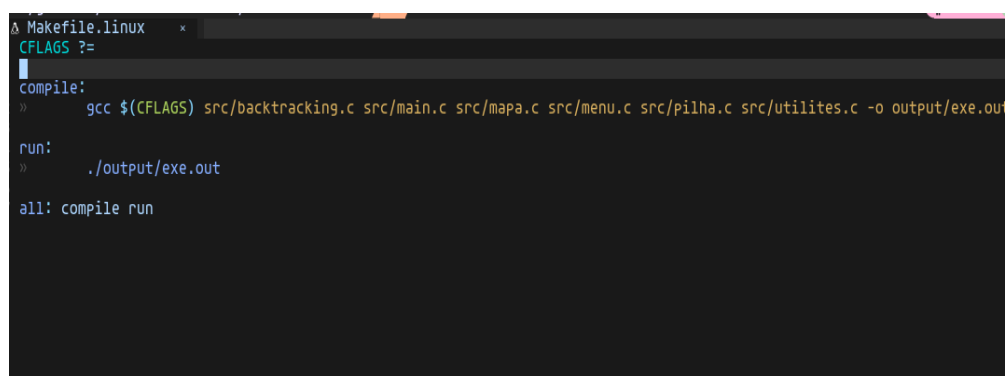
read -p "Entrar no modo análise? (s/n) -> s para sim / n para não: " resposta

if [[ "$resposta" == "s" || "$resposta" == "S" ]]; then
    echo "Entrando no modo análise ..."
    make -f Makefile.linux CFLAGS="-DANALISE"
else
    echo "Explorando labirinto normalmente ..."
    make -f Makefile.linux
fi

cd output
./exe.out
```

Figura 1: executa.sh

Como é possível notar na imagem acima, o comando make é executado com a flag “-f”, essa flag serve para buscar qual arquivo Makefile , dentro da pasta, será usado pelo programa (no caso Makefile.linux). Além disso, quando a entrada do usuário é “S” ou “s”, o comando make é executado utilizando as flags destinadas ao compilador (CFLAGS) para poder usar o “#define” do modo análise, do contrário o programa compila e executa normalmente. A figura 2 ilustra o corpo do arquivo de compilação do programa.

A screenshot of a text editor window titled 'Makefile.linux'. The content of the file is as follows:

```
CFLAGS ?=
compile:
» gcc $(CFLAGS) src/backtracking.c src/main.c src/mapa.c src/menu.c src/pilha.c src/utilites.c -o output/exe.out
run:
» ./output/exe.out
all: compile run
```

Figura 2: Makefile.linux

O arquivo consiste em receber ou não uma flag (para ativar o modo análise), então compila todos os arquivos com extensão.c (localizados na subpasta src) e cria um executável chamado exe.out na subpasta out.

Por fim, vale ressaltar que, na primeira vez que o arquivo for utilizado deve-se inserir o comando: “chmod +x executa.sh”, para conceder permissão de execução do arquivo executa.sh. Para executar o script, basta digitar no terminal, estando na pasta superior às subpastas src, headers e out, o comando “./executa.sh”.

Organização:

Mantivemos uma organização de diretórios bem definida, distribuída de acordo com os tipos de arquivos. No diretório raiz, temos presente as pastas de arquivos, headers, output, e src. Ainda no diretório raiz temos os arquivos .gitignore, executa.sh, makefile, Makefile.linux e o README.Md. A figura 3 apresenta a organização das pastas e arquivos usados para a confecção do trabalho prático.

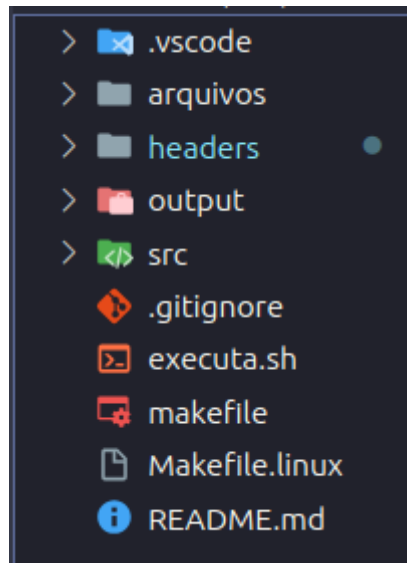


Figura 3: Diretório Raiz

/arquivos

Trata-se da pasta onde estão guardados os arquivos de entradas (Mapas/ Matrizes). É possível a adição de novos arquivos de entrada. OBS: Os arquivos devem ser do tipo .txt.

Desenvolvimento:

Para o desenvolvimento deste trabalho foram utilizados princípios SOLID de engenharia de software para se buscar maior clareza, responsabilidade única e uma maior legibilidade, visando alcançar futuras reutilizações ou ampliação do projeto. Para o mesmo, foram-se criadas subtarefas, as quais serão detalhadas a seguir.

Estruturas e Tipos de Dados Abstratos

O código fonte foi baseado em 4 principais estruturas de dados que desenvolvemos, sendo elas: o mapa, a pilha, aluno e coordenadas.

No TAD mapa, temos maneiras de lidar com tal , além de “typedefs” que facilitam o entendimento. Temos, lá presente um apontador de matrizes, chamado de “ApontadorMapa”, e um apontador de vetores, mais conhecido como matriz, mas a renomeamos como Mapa. A figura 4 ilustra o TAD Mapa.

```
//-----Renomeando tipos de dados-----
typedef int*** ApontadorMapa; // apontador de matriz de inteiros
typedef int** Mapa; // matriz de inteiros
```

Figura 4: Renomeação de novos tipos.

As estruturas coordenadas (presente no TAD coordenada) e Estudante (presente no TAD estudante), trabalham juntas, e servirão para manipularmos o estudante e correlaciona-lo à matriz (mapa). As figuras 5 e 6 ilustram os tipos abstratos de dados citados.

```
typedef struct coordenadas
{
    int x,y;
}
coordenadas;

//ponteiro pra estrutura de coordenadas
typedef coordenadas* ApontadorCoordenadas;
```

Figura 5: Estrutura e renomeação do TAD coordenadas .

```
typedef struct estudante
{
    int chaves_no_bolso;
    coordenadas pos_chave;
}
estudante;

//ponteiro pra estrutura de estudante
typedef estudante* ApontadorEstudante;
```

Figura 6: Estrutura e renomeação do TAD estudante.

Já todo o TAD pilha, tanto sua estrutura como manipulação foram essenciais para o desenvolvimento do backtracking - Abordaremos esse assunto mais abaixo. A figura 7 apresenta o TAD pilha.

```

#ifndef PILHA_H
#define PILHA_H

#include <stdio.h>
#include <stdlib.h>
#include "coordenadas.h"

typedef struct Pilha
{
    coordenadas* dimensao;
    int topo;
    int tamanhoPilha;
}
pilha;

//-----Funções da Pilha-----
pilha* criaPilha(int tamanhoPilha);
int pilhaVazia(pilha* stack);
coordenadas desempilha(pilha* stack);
void empilha(pilha* stack, int x, int y);
coordenadas olhaTopo(pilha* stack);
void destroiPilha(pilha* stack);

#endif

```

Figura 7: Estrutura e métodos de Pilha .

Mapa e Leitura

Para implementar a leitura de arquivos e construção do mapa, foi criada a função `InsererLabirinto()`, responsável por ler e processar os dados do arquivo. Além disso, outras funções foram criadas para a construção do labirinto no TAD Mapa.

Na função `InsererLabirinto()`, são recebidos como parâmetros os endereços para as estruturas de dados (TAD's) que armazenarão as informações necessárias para a construção do mapa do labirinto. Inicialmente, a função monta o caminho completo do arquivo e tenta abri-lo em modo de leitura. Caso a abertura falhe, uma mensagem de erro é exibida e a execução da função é encerrada. Em seguida, é realizada a leitura da primeira linha do arquivo, que contém os metadados necessários para a criação do mapa. Essas informações são armazenadas diretamente nas estruturas relacionadas. A figura 8 apresenta o corpo do método "`InsererLabirinto()`".

```

void InserirLabirinto(ApontadorMapa mapa, char Labirinto[], estudante* aluno, coordenadas* dimensao)
{
    FILE* labirinto;
    char caminhoArquivo[100];
    int linha = 0;
    int qtdLinhas, qtdColunas, qtdChaves;
    int mapaCarregado = 1;

    snprintf(caminhoArquivo, sizeof(caminhoArquivo), "../arquivos/%s.txt", Labirinto);
    //printf("%s\n", caminhoArquivo);

    labirinto = fopen(caminhoArquivo, "r");

    if (labirinto == NULL)
    {
        printf("Erro ao abrir o arquivo %s\n", Labirinto);
        return;
    }

    // Lê a primeira linha com os metadados
    if (fscanf(labirinto, "%d %d %d", &qtdLinhas, &qtdColunas, &qtdChaves) != 3)
    {
        printf("Erro ao ler os metadados do arquivo.\n");
        fclose(labirinto);
        return;
    }

    aluno->chaves_no_bolso = qtdChaves;
    dimensao->x = qtdLinhas;
    dimensao->y = qtdColunas;

    CriaMapa(mapa, qtdLinhas, qtdColunas);
}

```

figura 8: Função “InserirLabirinto()”

Após a etapa anterior, a função “CriaMapa()” é chamada. Esse método realiza a alocação dinâmica de memória para a matriz que representará o mapa, de acordo com as dimensões especificadas no arquivo. Primeiramente, é alocada memória para armazenar os ponteiros correspondentes às linhas da matriz. Em seguida, para cada linha da matriz, é alocada memória para as colunas correspondentes. Caso houver alguma falha, toda memória que foi alocada para a matriz é liberada. A figura 9 apresenta o corpo do método “CriaMapa()”.


```

void CriaMapa(ApontadorMapa mapa, int linhas, int colunas)
{
    *mapa = (int**)malloc(linhas * sizeof(int*)); // Aloca memória para as linhas
    if (*mapa == NULL)
    {
        printf("Erro ao alocar memória para as linhas do mapa\n");
        exit(1);
    }

    for (int i = 0; i < linhas; i++)
    {
        (*mapa)[i] = (int*)malloc(colunas * sizeof(int)); // Aloca memória para as colunas
        if ((*mapa)[i] == NULL)
        {
            printf("Erro ao alocar memória para as colunas do mapa\n");

            for (int j = 0; j < i; j++) // Libera linhas já alocadas
            {
                free((*mapa)[j]);
            }
            free(*mapa);
            exit(1);
        }
    }
}

```

figura 9: Método “CriaMapa()”

Depois da criação da matriz, a função “InserirLabirinto()” continua com o preenchimento do mapa utilizando os dados contidos no arquivo. Para isso, foi implementado um loop que percorre todas as linhas subsequentes. Para cada linha são convertidos os caracteres para números inteiros, que representam os diferentes elementos do mapa (como paredes e posição inicial). A figura 10 exibe o laço citado

```

for (int i = 0; i < qtdLinhas; i++)
{
    if (fgets(linhaArquivo, sizeof(linhaArquivo), labirinto) != NULL)
    {
        int valores[qtdColunas];
        for (int j = 0; j < qtdColunas; j++)
        {
            // Converte cada caractere da linha para o valor inteiro correspondente
            if (linhaArquivo[j] >= '0' && linhaArquivo[j] <= '4')
            {
                valores[j] = linhaArquivo[j] - '0';
            }else{
                printf("Erro ao ler uma posicao do labirinto\n");
                ApagaMapa(mapa, qtdLinhas);
                mapaCarregado = 0;
                break;
            }
        }

        PreencherMapa(mapa, i, valores, qtdColunas);
    }
    else
    {
        printf("Erro ao ler a linha %d do labirinto.\n", i + 1);
        ApagaMapa(mapa, qtdLinhas);
        mapaCarregado = 0;
        break;
    }
}

fclose(labirinto);

```

figura 10: Laço responsável pelo preenchimento do mapa

Durante a conversão, o algoritmo verifica se cada número corresponde a um valor válido no intervalo de 0 a 4, caso seja lido um valor inválido ou esteja faltando linhas do arquivo, a função interrompe o processo apagando o mapa que foi alocado dinamicamente através da função “ApagaMapa()”. A figura 11 exibe o corpo desta função.

```

void ApagaMapa(ApontadorMapa mapa, int linhas)
{
    if (*mapa == NULL)
        return;

    // Libera cada linha do mapa
    for (int i = 0; i < linhas; i++)
    {
        free((*mapa)[i]);
    }

    // Libera a matriz de ponteiros
    free(*mapa);

    // Define o ponteiro principal como NULL para evitar dangling pointers
    *mapa = NULL;
}

```

figura 11: Função “ApagaMapa()”

Caso não haja nenhum erro com a leitura dos dados, a função “PreencherMapa()” é executada, recebendo como parâmetros o ponteiro para o mapa, a posição da linha na qual os valores serão armazenados, o vetor contendo os valores que representam o conteúdo dessa linha e a quantidade de colunas que a matriz possui. Com base nesses parâmetros, a função utiliza um loop para iterar sobre cada coluna da linha especificada. Durante a iteração, os valores do vetor são copiados diretamente para a matriz, garantindo que os dados lidos do arquivo sejam corretamente armazenados na estrutura de dados que representa o mapa do labirinto. A figura 12 apresenta o corpo da função “PreencherMapa()”.

```

void PreencherMapa(ApontadorMapa mapa, int posicaoLinha, int valores[], int qtdColunas){
    for (int i = 0; i < qtdColunas; i++)
    {
        (*mapa)[posicaoLinha][i] = valores[i];
    }
}

```

figura 12: Método “PreencherMapa()”

Menu

Para a construção do menu, optou-se por uma chamada limpa no programa principal. Para isso, foi feito com que “menu.h” e “menu.c” apenas chamasse todas as funções dos TADs desenvolvidos. Portanto, o menu possui apenas uma instância no programa principal, evitando, assim, vazamentos de memória e a chamada dinamicamente alocada das estruturas que deva usar durante a execução do programa principal.

A função “logo()”, apresenta apenas uma ordem de caracteres a serem mostrados na tela, com a finalidade de quando vistos em uma alta granularidade o usuário entender como uma Logomarca do projeto, ela apresenta variação de 2 cores, apenas um detalhe visual chamativo. A figura 13 ilustra o corpo do método.

```
#include "../headers/menu.h"
#include "../headers/analise.h"

void logo(){
    printf("
    printf("aaa      aaaaaa aaaaaa aac aaaaaa aac aac aac aaaaaa aaaaaa \n");
    printf("aaa      aaaaaa aaaaaa aac aaaaaa aac aac aac aaaaaa aaaaaa \n");
    printf("aa! aa! aa! aa! aa! aa! aa! aa!aa!aa! aa! aa! aa! \n");
    printf("!@! @!@ !@ @!@ !@! @!@ !@! @!@!@! !@! @!@ \n");
    printf("@!@!@! @!@!@! !!@ @!@!@! !!@ @!@ !@! @!! @!@ \n");
    printf("!!! !!!!!! !!!!!! !!! !!@!@! !!! !@! !!! !!! !@! !!! \n");
    printf("!!: !!: !!: !!: !!: !!: !!: !!: !!: !!: !!: !!: !!: !!: !!: \n");
    printf(" :! :! :! :! :! :! :! :! :! :! :! :! :! :! :! \n");
    printf(" : : : : : : : : : : : : : : : : : : : : : : : : : : \n");
    printf(" : : : : : : : : : : : : : : : : : : : : : : : : : \n");
    printf("\033[0;37m"); //branco
}
```

Figura 13: Logomarca/ Nome do projeto.

Na função “menuPrincipal()” cria-se uma estrutura que consegue guardar um mapa por vez, o qual é um ponteiro para uma matriz, sendo os mapas sobrescritos e apagados na memória do heap. A figura 14 apresenta as definições das estruturas usadas pelo método “menuPrincipal()”.

```
ApontadorMapa guardaMapa = NULL;
Mapa mapa = NULL;
guardaMapa = &mapa;
estudante aluno;
coordenadas dimensao;
```

Figura 14: Estruturas que são passadas como parâmetro para a chamada das funções.

Essa função é responsável pela chamada de todas as outras chamadas, sendo essas acopladas e encapsuladas. A figura 15 ilustra as opções disponíveis para o usuário selecionar

```

6 }
7
8 void menuPrincipal(){
9     int opcao;
10
11     //instanciei o mapa, aluno e dimensoes
12     ApontadorMapa guardaMapa = NULL;
13     Mapa mapa = NULL;
14     guardaMapa = &mapa;
15
16     estudante aluno;
17     coordenadas dimensao;
18     //ApontadorMapa mapa = NULL; //inicialmente nao guarda mapa nenhum
19
20     do{
21         printf("\n\n");
22         printf("Opcoes do programa:\n");
23         printf("1) Carregar novo arquivo de dados.\n");
24         printf("2) Mostrar Graficamente o arquivo.\n");
25         printf("3) Processar e exibir resposta.\n");
26         printf("4) Gerar novo arquivo.\n");
27         printf("5) Backtracking com Aluno com poderes.\n");
28         printf("6) Encerrar o programa.\n");
29
30         printf("\nDigite um numero: ");
31         scanf("%d", &opcao);
32
33         switch(opcao){
34             case 1:
35                 CarregarArquivo(guardaMapa, &aluno, &dimensao);
36                 break;
37             case 2:
38                 VerGraficamente(guardaMapa, aluno, dimensao);
39                 break;
40             case 3:
41                 ProcessarDados(guardaMapa, aluno, dimensao);
42                 break;
43             case 4:
44                 GerarArquivo();
45                 break;
46             case 5:
47                 AlunoComPoderes(guardaMapa, aluno, dimensao);
48                 break;
49             case 6:
50                 printf("Saindo do sistema...\n");
51                 break;
52             default:
53                 limparTerminalUnix();
54                 printf("Opcao invalida! tente outra\n");
55         }
56     }while(opcao != 6);
57 }

```

figura 15: opções do usuário

Em suma, “opcao1()” permite o carregamento dos labirintos no programa, “opcao2()” realiza a operação de backtracking, “opcao3()” permite ao usuário criar um novo mapa para ser testado e a “opcao4()” encerra o programa. As figuras 16,17,18 e 19 exibem o corpo de cada um desses métodos citados.

```

1 void opcao1(ApontadorMapa mapa, ApontadorEstudante aluno, ApontadorCoordenadas dimensao){
2
3     //se o mapa nao existir, eu crio um novo
4     if (*mapa == NULL){
5
6         limparTerminalUnix();
7         printf("Por favor, digite o nome do arquivo sem extensao: ");
8         char nomeArquivo[50];
9         scanf("%s", nomeArquivo);
10
11         limparTerminalUnix();
12         InsereLabirinto(&(*mapa), nomeArquivo, &(*aluno), &(*dimensao));
13
14         printf("Deseja visualizar o mapa? (1 -> SIM, 0 -> NAO): ");
15         int opcao;
16         scanf("%d", &opcao);
17
18         if (opcao == 1){
19             printf("\n\n");
20             MostrarMapa((*mapa), (*dimensao).x, dimensao->y);
21             pressEnter();
22             limparTerminalUnix();
23             return;
24         }
25         else return;
26     }
27
28     //se o mapa ja existir, eu apago o que ja existe e crio um novo
29     if (*mapa != NULL)
30     {

```

figura 16: “opcao1()”, se não houver nenhum mapa previamente carregado

```

31         //se o mapa ja existir, eu apago o que ja existe e crio um novo
32         if (*mapa != NULL)
33         {
34             limparTerminalUnix();
35             printf("Por favor, digite o nome do arquivo sem extensao: ");
36             char nomeArquivo[50];
37             scanf("%s", nomeArquivo);
38
39             limparTerminalUnix();
40             ApagaMapa(&(*mapa), (*dimensao).x); //DIFERENÇA QUE TA APAGANDO O MAPA ANTERIOR DO HEAP E NAO SO O
41             InsereLabirinto(&(*mapa), nomeArquivo, &(*aluno), &(*dimensao));
42
43             printf("Deseja visualizar o mapa? (1 -> SIM, 0 -> NAO): ");
44             int opcao;
45             scanf("%d", &opcao);
46
47             if (opcao == 1){
48                 printf("\n\n");
49                 MostrarMapa((*mapa), (*dimensao).x, dimensao->y);
50                 pressEnter();
51                 limparTerminalUnix();
52                 return;
53             }
54             else return;
55         }
56     }

```

figura 17: “opcao1()”, se houver mapa já carregado

```

1 void opcao2(ApontadorMapa guardaMapa, estudante aluno, coordenadas dimensao) {
2     limparTerminalUnix();
3     if (*guardaMapa != NULL) {
4         #ifndef ANALISE
5             ExploraLabirinto(*guardaMapa, dimensao.x, dimensao.y, aluno);
6             ApagaMapa(guardaMapa, dimensao.x);
7         #else
8             ExploraLabirinto(*guardaMapa, dimensao.x, dimensao.y, aluno);
9             resultadoAnalise();
10            ApagaMapa(guardaMapa, dimensao.x);
11        #endif
12        return;
13    } else {
14        printf("Sem labirinto a ser explorado! Carregue um arquivo.\n");
15        return;
16    }
17 }

```

figura 18: “opcao2()”, executa algoritmo de backtracking, caso haja mapa carregado

```

1 void opcao3(){
2     int largura, altura, chaves, dificuldade, chavesPerdidas;
3     char nomeArquivo[50];
4
5     printf("Digite o nome do arquivo sem extensao: ");
6     scanf("%s", nomeArquivo);
7     printf("Digite a largura do labirinto: ");
8     scanf("%d", &largura);
9     printf("Digite a altura do labirinto: ");
10    scanf("%d", &altura);
11    printf("Digite a quantidade de chaves iniciais: ");
12    scanf("%d", &chaves);
13    printf("Digite a dificuldade (0-100, onde valores altos geram mais paredes): ");
14    scanf("%d", &dificuldade);
15    printf("Digite quantas chaves estão perdidas no labirinto: ");
16    scanf("%d", &chavesPerdidas);
17
18    gerarLabirinto(nomeArquivo, largura, altura, chaves, dificuldade, chavesPerdidas);
19
20    return;
21 }

```

figura 19: “opcao3()”, entradas de dados para executar função “geraLabirinto()”

Backtracking

Para a implementação do backtracking, foram criadas duas funções: “ExploraLabirinto()” e “movimenta_estudante()”.

Primeiramente, no método “ExploraLabirinto()”, é determinada a posição de partida, ou seja, em qual posição do mapa o estudante se encontra. Após isso, é criada uma pilha, na qual serão armazenadas as coordenadas de cada movimento feito pelo estudante, incluindo sua posição inicial e, em seguida, define-se a ordem dos movimentos a serem realizados - passo importante para o backtracking - através

de dois vetores. Tais vetores, possuem cada um quatro elementos, cujo par(x,y) representa o tipo de movimento a ser realizado, ou seja, para cima, para baixo, para a esquerda ou para a direita. Em seguida, é feita uma chamada a função recursiva “movimenta_estudante()”, a qual chama a si própria até que o estudante chegue na primeira linha (linha zero da matriz).

```
void ExploraLabirinto(Mapa mapa, int linhas, int colunas, estudante aluno){
    coordenadas posicao;
    //Determina ponto de partida do mapa
    for(int i = 0; i < linhas; i++){
        for(int j = 0; j < colunas; j++){
            if(mapa[i][j] == 0){
                posicao.x = i;
                posicao.y = j;
                break;
            }
        }
    }
    coordenadas dimensao = {linhas, colunas};
    pilha* stack = criaPilha(dimensao.x * dimensao.y);
    empilha(stack, posicao.x, posicao.y);
    printf("Linha: %d Coluna: %d\n", posicao.x, posicao.y);

    int moveX[] = {-1,1,0,0};
    int moveY[] = {0,0,-1,1};

    int chamadasRecursivas = 0;

    if (movimenta_estudante(mapa, aluno, dimensao, &chamadasRecursivas, stack, moveX, moveY, posicao) == -1){
        printf("\nO estudante se moveu %d vezes e percebeu que o labirinto não tem saída!\n", chamadasRecursivas);
        destroiPilha(stack);
        return;
    }
}
```

Figura 20: Implementação da função “ExploraLabirinto()”.

Na função “movimenta_estudante()”, em primeiro lugar, verifica-se se a pilha está vazia, pois, tal fator é fundamental para que se determine o fim das chamadas recursivas e determinar que o labirinto não tem saída. Em seguida, a posição onde o estudante passou no labirinto é sinalizada, para que se registre os caminhos que foram tentados e não os tente novamente. Além disso, tenta-se realizar um dos quatro movimentos possíveis (para cima, para baixo, para a esquerda e para a direita) e, caso seja possível, esse movimento é empilhado. Caso nenhuma das quatro possibilidades possa ser realizada, é desempilhado o último movimento realizado pelo estudante, simbolizando, através de uma estrutura de pilha, o retorno na árvore de decisão, a qual é comum em backtracking. Essa função é chamada recursivamente até que se chegue à primeira linha do labirinto ou até que a pilha fique vazia, indicando que não há mais movimentos possíveis a serem tentados e, por consequência, que não há uma solução para o problema, ou seja, é impossível que o estudante chegue na linha zero do labirinto.


```

int movimenta_estudante(Mapa mapa, estudante aluno, coordenadas dimensao, int *cont,
    pilha* stack, int *moveX, int *moveY, coordenadas posicao){

    if (pilhaVazia(stack)){
        return -1;
    }

    int moveu;
    posicao.x = olhaTopo(stack).x;
    posicao.y = olhaTopo(stack).y;

    //passo base, se chegou na linha 0. Sucesso saiu do labirinto
    if(posicao.x == 0){
        printf("\n0 estudante se moveu %d vezes e chegou na coluna %d da primeira linha\n", (*cont), posicao.y);

        destroiPilha(stack);
        return 0;
    }

    //Marca onde passou no labirinto
    mapa[posicao.x][posicao.y] = -1;
    moveu = 0;

    for(int i = 0; i < 4; i++){
        int x_atual = posicao.x + moveX[i];
        int y_atual = posicao.y + moveY[i];
        //Chamadas recursivas
        #ifdef ANALISE
            contagemRecursiva++;
        #endif

        int ehMovimentavel = 0;

        if(x_atual >= 0 && y_atual >= 0 && x_atual < dimensao.x && y_atual < dimensao.y){

            //Se caminho livre ou se porta e estudante com chave -> movimento permitido
            if(mapa[x_atual][y_atual] == 1 || mapa[x_atual][y_atual] == 4 || (mapa[x_atual][y_atual] == 3 && aluno.chaves_no_bolso >= 1)){

```

Figura 21: Implementação da função “movimenta_estudante()” parte 1.

```

        if(x_atual >= 0 && y_atual >= 0 && x_atual < dimensao.x && y_atual < dimensao.y){

            //Se caminho livre ou se porta e estudante com chave -> movimento permitido
            if(mapa[x_atual][y_atual] == 1 || mapa[x_atual][y_atual] == 4 || (mapa[x_atual][y_atual] == 3 && aluno.chaves_no_bolso >= 1)){
                if(mapa[x_atual][y_atual] == 3) aluno.chaves_no_bolso--;
                if(mapa[x_atual][y_atual] == 4)
                {
                    aluno.chaves_no_bolso++;
                    aluno.pos_chave.x = x_atual;
                    aluno.pos_chave.y = y_atual;
                }
                printf("Linha: %d Coluna: %d\n", x_atual, y_atual);
                ehMovimentavel = 1;
            }
        }
        if(ehMovimentavel){
            empilha(stack, x_atual, y_atual);
            moveu = 1;

            //Recursividade máxima
            #ifdef ANALISE
                break;
            #endif
        }
    }

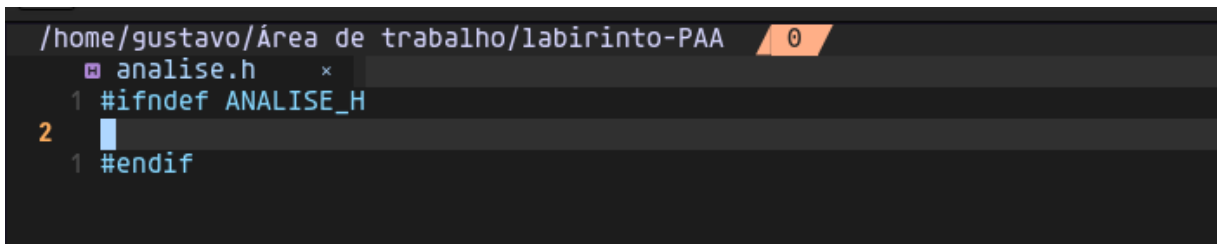
    if(!moveu){
        desempilha(stack);
        if(!pilhaVazia(stack)){
            coordenadas topo = olhaTopo(stack);
            if(topo.x == aluno.pos_chave.x && topo.y == aluno.pos_chave.y && aluno.chaves_no_bolso > 0){
                aluno.chaves_no_bolso--;
                mapa[aluno.pos_chave.x][aluno.pos_chave.y] = 4;
                printf("Aluno devolveu a chave na posição (%d %d)\n", aluno.pos_chave.x, aluno.pos_chave.y);
            }
            printf("Linha: %d Coluna: %d\n", topo.x, topo.y);
        }
    }
    (*cont)++;
    return movimenta_estudante(mapa, aluno, dimensao, cont, stack, moveX, moveY, posicao);
}

```

Figura 22: Implementação da função “movimenta_estudante()” parte 2

Modo análise

O modo é ativado somente se o usuário entrar com “S” ou “s” após executar o arquivo `executa.sh`. Como indicado na especificação, esse método deve ser capaz de realizar tudo o que o programa faz normalmente, mais avaliar o nível máximo de recursividade e a contagem de chamadas recursivas. Em primeiro lugar, temos o arquivo “`analise.h`” que declara o “`#define`” para ser usado no decorrer do programa. A figura 23 ilustra o corpo do arquivo.



```
/home/gustavo/Área de trabalho/labirinto-PAA 0
analise.h x
1 #ifndef ANALISE_H
2
1 #endif
```

figura 23: `analise.h`

Consecutivamente, o “`#define`” é usado dentro do arquivo “`backtracking.c`”, mais especificamente no método “`movimenta_estudante()`”.

A contagem de chamadas recursivas é feita na tentativa de movimentação do estudante dentro do labirinto. Essa abordagem foi escolhida, pois é a quantidade de possíveis movimentações feitas pelo aluno. Como a operação relevante do algoritmo é a tentativa de movimento do estudante a cada nova chamada, é compreensível contabilizar as chamadas recursivas neste ponto do código e não ao final deste (etapa de retornar na árvore de possibilidades).

Ademais, o nível máximo de recursividade faz uso direto da estrutura de pilha, pois ao conferir o tamanho máximo que a pilha assume, é possível conferir quantas vezes o estudante conseguiu avançar no labirinto ininterruptamente. Em suma, o algoritmo define o tamanho máximo da pilha, ao conferir o atributo `topo` da estrutura de pilha, um inteiro que indica o tamanho da pilha. Se “`pos_topo`” for menor “`stack->topo`”, então “`pos_topo`” se torna o valor “`stack->topo`”. A figura 24 ilustra o que foi explicado de forma mais elaborada até então.

```

int movimenta_estudante(Mapa mapa, estudante aluno, coordenadas dimensao, int *cont,
    pilha* stack, int *moveX, int *moveY, coordenadas posicao){
    if (!pilhaVazia(stack)){-
        int moveu;
        posicao.x = olhaTopo(stack).x;
        posicao.y = olhaTopo(stack).y;
        //passo base, se chegou na linha 0. Sucesso saiu do labirinto
        if(posicao.x == 0){-
            //Marca onde passou no labirinto
            mapa[posicao.x][posicao.y] = -1;
            moveu = 0;
            for(int i = 0; i < 4; i++){
                int x_atual = posicao.x + moveX[i];
                int y_atual = posicao.y + moveY[i];
                //Chamadas recursivas
                #ifdef ANALISE
                    contagemRecursiva++;
                #endif
                int ehMovimentavel = 0;
                if(x_atual >= 0 && y_atual >= 0 && x_atual < dimensao.x && y_atual < dimensao.y){
                    //Se caminho livre ou se porta e estudante com chave -> movimento permitido
                    if(mapa[x_atual][y_atual] == 1 || mapa[x_atual][y_atual] == 4 || (mapa[x_atual][y_atual] == 3 && aluno.chaves_no_bolso >= 1)){-
                        if(ehMovimentavel){
                            empilha(stack, x_atual, y_atual);
                            moveu = 1;
                            //Recursividade máxima
                            #ifdef ANALISE
                                topo = olhaTopo(stack);
                                if(contagemRecursiva > 1)
                                    if(pos_topo < stack->topo) pos_topo = stack->topo;
                                else pos_topo = stack->topo;
                                auxiliar = criaPilha(dimensao.x * dimensao.y);
                                while(!pilhaVazia(stack))
                                    {
                                        coordenadas aux = desempilha(stack);
                                        empilha(auxiliar, aux.x, aux.y);
                                    }
                                while(!pilhaVazia(auxiliar))
                                    {
                                        coordenadas aux = desempilha(auxiliar);
                                        empilha(stack, aux.x, aux.y);
                                    }
                                destroiPilha(auxiliar);
                            #endif
                            break;
                        }
                    }
                }
            }
        }
    }
}

```

figura 24: Uso do modo análise

Para poder exibir os resultados obtidos pelo modo análise, foi implementada a função “resultadoAnalise()” que apenas imprime no terminal a quantidade de chamadas recursivas e o nível máximo de recursividade, se “#define” estiver ativo. Vale ressaltar que o valor 0 atribuído a contagem serve para impedir que o resultado obtido em um mapa influencie no outro. A figura 25 ilustra o corpo dessa função.

```

void resultadoAnalise()
{
    #ifdef ANALISE
        printf("chamadas recursivas = %d\n", contagemRecursiva);
        printf("nivel max = %d\n", pos_topo+1);
        contagemRecursiva = 0;
    #endif
}

```

figura 25: “resultadoAnalise()”

Por fim, o último local que o “#define” é utilizado é dentro da função “opcao2()” que invoca o resultado obtido pelo mapa carregado. Se “#define” ativo, usa o modo análise e imprime o resultado no terminal, do contrário executa no modo normal. A figura 26 exibe este comportamento.

```

void opcao2(Mapa mapa,estudante aluno,coordenadas dimensao){
    limparTerminalUnix();
    if(*mapa != NULL){
        #ifndef ANALISE
            ExploraLabirinto(mapa, dimensao.x, dimensao.y, aluno);
        #else
            ExploraLabirinto(mapa, dimensao.x, dimensao.y, aluno);
            resultadoAnalise();
        #endif
        return;
    }
    else {
        printf("Sem labirinto a ser explorado! Tente novamente\n");
        return;
    }
    return;
}

```

figura 26: comportamento do “#define” na função “opcao2()”

Extras

Ponto extra 3:

A elaboração dessa tarefa consistia em permitir que o estudante, conforme andasse pelo labirinto, fosse capaz de pegar chaves pelo chão (marcadas pelo número 4) e, caso voltasse para a mesma posição, deveria devolvê-la, caso esta

não tivesse sido usada. Como o algoritmo de tentativa e erro, elaborado pelo grupo, já faz uso de uma estrutura de pilha, bastou modificar a estrutura “estudante” para esta poder marcar a posição onde a chave foi coletada e adicionar mais algumas condições ao código. A figura 27 ilustra a condição extra adicionada ao algoritmo original de backtracking

```
int ehMovimentavel = 0;

if(x_atual >= 0 && y_atual >= 0 && x_atual < dimensao.x && y_atual < dimensao.y){
    //Se caminho livre ou se porta e estudante com chave -> movimento permitido
    if(mapa[x_atual][y_atual] == 1 || mapa[x_atual][y_atual] == 4 || (mapa[x_atual][y_atual] == 3 && aluno.chaves_no_bolso > 1)){
        if(mapa[x_atual][y_atual] == 3) aluno.chaves_no_bolso--;
        if(mapa[x_atual][y_atual] == 4)
        {
            aluno.chaves_no_bolso++;
            aluno.pos_chave.x = x_atual;
            aluno.pos_chave.y = y_atual;
        }
        printf("Linha: %d Coluna: %d\n", x_atual, y_atual);
        ehMovimentavel = 1;
    }
}
```

Figura 27: condição que verifica se há chave na posição atual do estudante.

Em outras palavras, se a posição atual do estudante fosse diferente de dois e se o estudante possuísse ao menos 1 chave, ou seja, a posição atual for 3, esta posição seria, portanto, permitida. Se o movimento for possível para esta posição é preciso conferir se é uma porta (marcada pelo número 3) ou se é o local onde uma chave foi deixada. Na situação de ser uma porta, o atributo “chaves_no_bolso” é decrementado e na situação de ser uma chave no chão, este atributo é incrementado e, além disso, o atributo “pos_chave” marca a posição em que a chave foi coletada.

Como se sabe, caso não seja possível avançar a partir de uma dada posição no algoritmo de backtracking, é preciso retornar à nós anteriores de sua árvore de decisão, representada neste trabalho por uma estrutura de pilha. A figura 28 exhibe a modificação do algoritmo que simula o retorno nos nós da árvore, ou seja, desempilhar.

```
if(!moveu){
    desempilha(stack);
    if(!pilhaVazia(stack)){
        coordenadas topo = olhaTopo(stack);
        if(topo.x == aluno.pos_chave.x && topo.y == aluno.pos_chave.y && aluno.chaves_no_bolso > 0){
            mapa[aluno.pos_chave.x][aluno.pos_chave.y] = 4;
            aluno.chaves_no_bolso--;
            printf("Aluno devolveu a chave na posição (%d %d)\n", aluno.pos_chave.x, aluno.pos_chave.y);
        }
        printf("Linha: %d Coluna: %d\n", topo.x, topo.y);
    }
}
(*cont)++;
return movimenta_estudante(mapa, aluno, dimensao, cont, stack, moveX, moveY, posicao);
```

Figura 28: condicional para devolver chave e voltar na pilha

Ao realizar a função “desempilha()”, é conferido se o que está sendo desempilhado é a posição onde a chave foi deixada, se positivo, a posição é marcada com o número 4, caso o estudante ainda tenha chaves no bolso, do contrário, só executa o passo comum do algoritmo de backtracking.

Ponto extra 2:

O desenvolvimento desse ponto extra consiste na implementação de uma funcionalidade capaz de gerar novos arquivos de teste. Para tal, foi criada a função “geraLabirinto()” que a partir dos valores de entrada passados pelo usuário (nome do arquivo de teste a ser gerado, dimensões do mapa, quantidade de chaves que o estudante começa, limiar que determina a quantidade de paredes no labirinto e quantidade de chaves distribuídas no labirinto), produz um novo labirinto personalizado. Assim como no trabalho prático 0 da disciplina, o algoritmo para geração de labirintos também faz uso da aleatoriedade, sendo esta usada para determinar a composição do labirinto (quantidade de passagens, portas, chaves no chão, paredes e posição inicial do estudante). Vale destacar que para gerar as paredes (colorir labirinto de azul) utiliza-se o limiar passado como entrada na chamada da função, se este for superior a 10, paredes podem ser geradas, senão, será uma porta ou uma passagem. As figuras 29, 30 exibem o processo de coloração do novo labirinto.

```

// Semente para geração de números aleatórios
srand(time(NULL));

// Matriz para o labirinto
int labirinto[altura][largura];

// Escolher aleatoriamente a coluna do valor VERDE na última linha
int colunaVerde = rand() % largura;

// Gerar as células do labirinto
for (int i = 0; i < altura; i++) {
    for (int j = 0; j < largura; j++) {
        if (i == altura - 1 && j == colunaVerde) {
            labirinto[i][j] = VERDE; // Célula inicial
        } else {
            int prob = rand() % 100;
            if (prob < dificuldade) {
                labirinto[i][j] = AZUL; // Parede
            } else if (prob < dificuldade + 10) {
                labirinto[i][j] = VERMELHO; // Porta
            } else {
                labirinto[i][j] = BRANCO; // Espaço vazio
            }
        }
    }
}

```

figura 29: coloração do labirinto de verde, azul, vermelho e branco

```

// Colocar as chaves (AMARELO) em posições aleatórias
for (int c = 0; c < chavesPerdidas; c++) {
    int x, y;
    do {
        x = rand() % altura;
        y = rand() % largura;
    } while (labirinto[x][y] != BRANCO); // Garantir que a chave seja colocada em uma célula vazia

    labirinto[x][y] = AMARELO; // Posicionar a chave
}

// Escrever o labirinto no arquivo
for (int i = 0; i < altura; i++) {
    for (int j = 0; j < largura; j++) {
        fprintf(arquivo, "%d", labirinto[i][j]);
    }
    fprintf(arquivo, "\n");
}

```

figura 30: coloração do labirinto de amarelo e escrita do labirinto no arquivo a ser gerado

Ponto extra 1:

Para evitar redundância, as implementações alternativas de interface desenvolvidas pelo grupo estão sendo apresentadas nas seções “Menu” e “Resultados”.

Ponto Extra 5:

Para o calcular de Complexidade do algoritmo, usei a biblioteca Time.h,

salvando o tempo imutável em uma variável ao incio do backtraking e salvando o tempo novamente em outra ao final de sua execução, fazendo um cálculo (final - inicial) para sabermos de fato o tempo gasto pelo algoritmo...

5 entradas distintas foram usadas como base, sendo elas N , e N² sendo o tamanho quadrático da matriz.

Usamos matrizes quadráticas de 10, 20, 100, 200, e 100. Onde a dificuldade delas foram settadas em 50%, sem chaves perdidas no labirinto e 20% de N o número de chaves com aluno.

OBS: O tempo de Execução só é visível no modo análise

```
0 estudante se moveu 5 vezes e percebeu que o labirinto não tem saída!  
chamadas recursivas = 18  
nivel max = 3  
Tempo de execução: 0.00002 segundos
```

Figura :Matriz T1 10x10

```
0 estudante se moveu 15 vezes e percebeu que o labirinto não tem saída!  
chamadas recursivas = 51  
nivel max = 6  
Tempo de execução: 0.00002 segundos
```

Figura :Matriz T2 20x20

```
0 estudante se moveu 31 vezes e percebeu que o labirinto não tem saída!  
chamadas recursivas = 98  
nivel max = 13  
Tempo de execução: 0.00002 segundos
```

Figura :Matriz T1 100x100

```
0 estudante se moveu 15 vezes e percebeu que o labirinto não tem saída!  
chamadas recursivas = 51  
nivel max = 13  
Tempo de execução: 0.00003 segundos
```

Figura :Matriz T1 200x200


```
0 estudante se moveu 55 vezes e percebeu que o labirinto não tem saída!  
chamadas recursivas = 180  
nível max = 21  
Tempo de execução: 0.00004 segundos
```

Figura :Matriz T1 1000x1000

Para Matrizes qualquer entrada de matrizes , tesmo as seguintes complexidades :

- **matriz quadratica: $O(n^2)$**
- **matrizes: $O(n*m)$** onde n e m são os números de altura e coluna respectivamente

A função $T(N)$ para nosso algoritmo backtracking, levamos em conta o número de comparações realizadas. Para cada ,célula na matriz, portanto temos que realizar 4 verificações, levando então até a fórmula : **$T(n,m) = 4*n*m$** .

O grafico de tempo X Entradas (tempo: eixo y e Entradas eixo x) plotado apresenta-se abaixo:

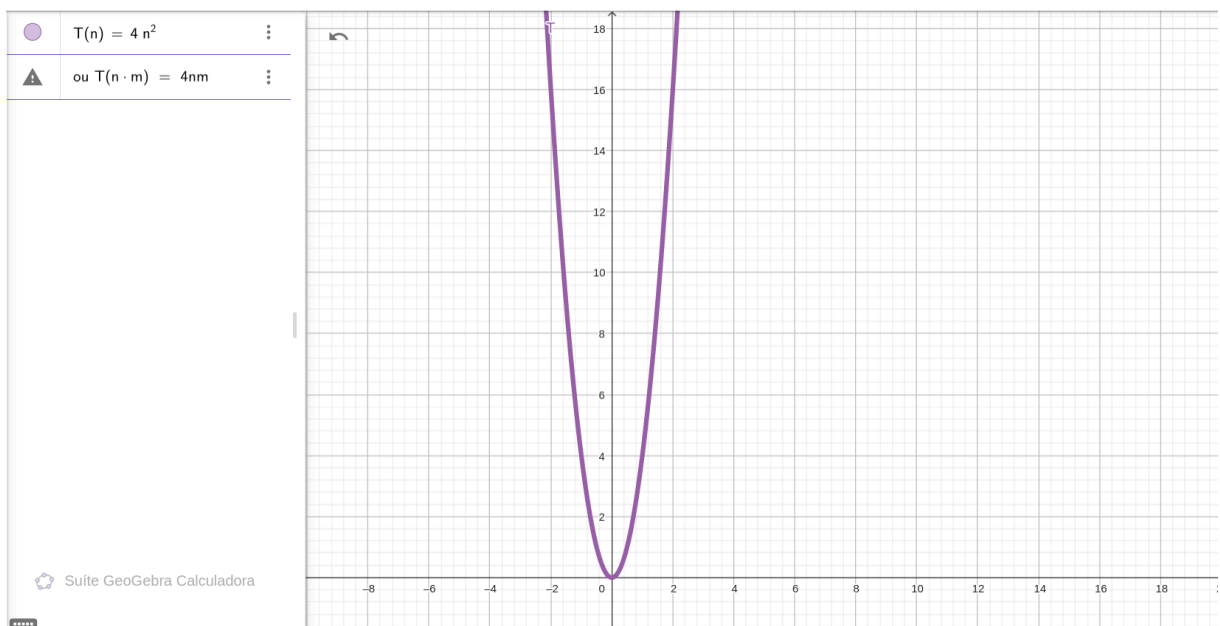


Figura : Gráfico plotado, $T(N)$

Ponto Extra 4:

Nossa parte criativa visa um aluno que possui cargas de poderes, que quando ele não tenha acesso a portas, chaves ou caminhos ele possa usá-los na finalidade de atravessar qualquer obstáculo.

```

267
268 int ehMovimentavel = 0;
269
270 //conferindo se não o movimento não está marcado e se ele está dentro das dimensões do mapa
271 if(x_atual >= 0 && y_atual >= 0 && x_atual < dimensao.x && y_atual < dimensao.y){
272
273     //Se caminho livre ou se porta e estudante com chave -> movimento permitido e usando cargas de poderes para passar por portas
274     if(mapa[x_atual][y_atual] == 1 || mapa[x_atual][y_atual] == 4 ||
275        (mapa[x_atual][y_atual] == 3 && aluno.chaves_no_bolso >= 1) ||
276        (mapa[x_atual][y_atual] == 3 && aluno.chaves_no_bolso <= 1 && poderes > 0) ||
277        (mapa[x_atual][y_atual] == 2 && aluno.chaves_no_bolso <= 1 && poderes > 0)){
278         if(mapa[x_atual][y_atual] == 3) aluno.chaves_no_bolso--;
279         if(mapa[x_atual][y_atual] == 2 && aluno.chaves_no_bolso <= 1 && poderes > 0) poderes--;
280         if(mapa[x_atual][y_atual] == 3 && aluno.chaves_no_bolso <= 1 && poderes > 0) poderes--;
281         if(mapa[x_atual][y_atual] == 4)
282         {
283             aluno.chaves_no_bolso++;
284             aluno.pos_chave.x = x_atual;
285             aluno.pos_chave.y = y_atual;
286         }
287         printf("Linha: %d Coluna: %d\n",x_atual,y_atual);
288         ehMovimentavel = 1;
289     }
290 }
291 if(ehMovimentavel){
292     empilha(stack, x_atual, y_atual);
293     moveu = 1;
294
295     break;
296 }

```

Figura : Tratamento condicional de poderes

A cada vez que ele chega a uma porta, ou parede e não tenha chaves para atravessar , o aluno usará uma carga de seu poder... Isso facilitará na chegada do fim do labirinto, embora o aluno também possa se cansar (esgotar poderes) e ficar ainda mais encurralado.

Resultados

A seguir serão exibidas imagens dos arquivos testados e seus respectivos resultados:

```

Mapa carregado com sucesso.

Deseja visualizar o mapa? (1 -> SIM, 0 -> NAO): 1

1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
3 2 2 2 2 2 2 1 1 1
1 1 1 1 1 1 2 1 1 1
1 2 2 2 2 2 2 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 0 1 1 1 1 1

Pressione Enter para continuar...

```

Figura 31: Arquivo de teste “labirinto3.txt”.

```
Linha: 3 Coluna: 0
Linha: 3 Coluna: 1
Linha: 2 Coluna: 1
Linha: 2 Coluna: 2
Linha: 1 Coluna: 2
Linha: 0 Coluna: 2

0 estudante se moveu 5 vezes e chegou na coluna 2 da primeira linha
chamadas recursivas = 11
nivel max = 48
```

Figura 32: Saída obtida com a execução do arquivo “labirinto3.txt” em modo análise.

```
Mapa carregado com sucesso.

Deseja visualizar o mapa? (1 -> SIM, 0 -> NAO): 1

1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 1 2 2 2 2
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
3 2 2 2 2 2 2 1 1 1
1 1 1 1 1 1 2 1 1 1
1 2 2 2 2 2 2 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 0 1 1 1 1 1

Pressione Enter para continuar...
```

Figura 33: Arquivo de teste “labirinto5.txt”.

```
Linha: 4 Coluna: 2
Linha: 3 Coluna: 2
Linha: 3 Coluna: 3
Linha: 4 Coluna: 3
Linha: 4 Coluna: 4
Linha: 3 Coluna: 4
Linha: 3 Coluna: 5
Linha: 2 Coluna: 5
Linha: 1 Coluna: 5
Linha: 0 Coluna: 5

0 estudante se moveu 26 vezes e chegou na coluna 5 da primeira linha
chamadas recursivas = 53
nivel max = 27
```

Figura 34: Parte da saída obtida com a execução do arquivo “labirinto5.txt” em modo análise.

```

Mapa carregado com sucesso.

Deseja visualizar o mapa? (1 -> SIM, 0 -> NAO): 1

1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
3 2 2 2 2 2 2 1 1 1
1 1 1 1 1 1 2 1 1 1
1 2 2 2 2 2 2 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 0 1 1 1 1 1

Pressione Enter para continuar...

```

Figura 35: Arquivo de teste “labirinto.txt”.

```

Linha: 7 Coluna: 0
Linha: 8 Coluna: 0
Linha: 9 Coluna: 0
Linha: 9 Coluna: 1
Linha: 8 Coluna: 1
Linha: 8 Coluna: 2
Linha: 9 Coluna: 2
Linha: 9 Coluna: 3
Linha: 8 Coluna: 3
Linha: 8 Coluna: 4
Linha: 9 Coluna: 4

O estudante se moveu 113 vezes e percebeu que o labirinto não tem saída!
chamadas recursivas = 356
nível max = 48

```

Figura 36: Parte da saída obtida com a execução do arquivo “labirinto.txt” em modo análise.

```

Mapa carregado com sucesso.

Deseja visualizar o mapa? (1 -> SIM, 0 -> NAO): 1

2 1 1 2
3 1 1 2
1 2 2 3
0 4 4 1

Pressione Enter para continuar...

```

Figura 37: Arquivo de teste “labirinto4.txt” - Ponto extra 3.

```

Linha: 2 Coluna: 0
Linha: 3 Coluna: 0
Linha: 3 Coluna: 1
Linha: 3 Coluna: 2
Linha: 3 Coluna: 3
Linha: 2 Coluna: 3
Linha: 3 Coluna: 3
Aluno devolveu a chave na posição (3 2)
Linha: 3 Coluna: 2
Linha: 3 Coluna: 1
Linha: 3 Coluna: 0

0 estudante se moveu 11 vezes e percebeu que o labirinto não tem saída!
chamadas recursivas = 38
nivel max = 5

```

Figura 38: Parte da saída obtida com a execução do arquivo “labirinto4.txt” em modo análise.

Conclusão

Este trabalho teve como objetivo implementar um algoritmo de backtracking para explorar os caminhos possíveis em um labirinto, buscando uma solução viável ou indicando que, após o esgotamento de todas as tentativas, não é possível alcançar o objetivo final: atingir a linha 0 do labirinto. Durante o desenvolvimento, enfrentamos diversas dificuldades, desde a modelagem inicial até ajustes na lógica para lidar com casos mais complexos. No entanto, essas adversidades foram essenciais para o aprendizado, permitindo-nos aprimorar nossas habilidades em resolução de problemas e fortalecer nossa compreensão do algoritmo. Consideramos, assim, que a execução deste trabalho foi **bem-sucedida**, dado os resultados satisfatórios obtidos e o conhecimento adquirido ao longo do processo.

Observações:

O programa foi desenvolvido para ser executado no sistema operacional Linux e o terminal deve preferencialmente possuir o shell script BASH.

Referência:

FNKY. **ANSI Escape Codes**. 2024. Disponível em:

<https://gist.github.com/fnky/458719343aabd01cfb17a3a4f7296797>. Acesso em: 29 nov. 2024.

STACKOVERFLOW. **CFLAGS, CCFLAGS, CXXFLAGS — what exactly do these variables control?** 2011. Disponível em:

<https://stackoverflow.com/questions/5541946/cflags-ccflags-cxxflags-what-exactly-do-these-variables-control>. Acesso em: 3 dez. 2024.

CAIORSS. **Compiler Flags and Options**. 2024. Disponível em:

<https://caiorss.github.io/C-Cpp-Notes/compiler-flags-options.html>. Acesso em: 3 dez. 2024.

GNU. **Bash Reference Manual**. 2024. Disponível em:

<https://www.gnu.org/savannah-checkouts/gnu/bash/manual/bash.html>. Acesso em: 3 dez. 2024.